```
//
// ============================================================================
//  FILE: HUDManager.cs
//  PATH: Engine/UI/
//  MODULE: HUD Manager
//
//  ROLE:
//      Central manager for HUD layout loading, rendering, and state management.
Integrates the new
//      HUDPanel Finalizer pipeline (Manager → Control → ColorParser → Resolver →
UIStateBuilder)
//      and coordinates all HUDPanel_* runtime elements.
//
//  RESPONSIBILITIES:
//      - Load HUD layout JSON via HUDConfigManager.
//      - Manage HUD textures, text elements, and render order.
//      - Maintain hitboxes for cash/lives increment buttons.
//      - Update HUD text values based on PlayerSystem state.
//      - Draw HUD elements using IDrawingContext.
//      - Integrate HUDPanelFinalizer_Manager for deterministic CASH panel configuration.
//      - Distribute resolved finalizer colors to HUDPanel_CashUpdate.
//
//  NON-RESPONSIBILITIES:
//      - Finalizer pipeline logic (handled by HUDPanelFinalizer_* modules).
//      - Rendering context creation.
//      - Player state management.
//      - Snapshot persistence.
//
//  NOTES:
//      This file has been fully modernized to remove legacy HUDPanel_Finalizer
references and now
//      uses HUDPanelFinalizer_Manager exclusively. All type mismatches, accessibility
issues, and
//      rendering context inconsistencies have been resolved.
//
//
// ============================================================================
using System;
using System.Collections.Generic;
using System.Drawing;
using System.IO;
using System.Linq;
using System.Text.Json;
using SASZombieAssaultTD.Engine.Diagnostics;
using SASZombieAssaultTD.Engine.Input;
using SASZombieAssaultTD.Engine.Player;
using SASZombieAssaultTD.Engine.Rendering;
using SASZombieAssaultTD.Engine.Snapshot;
using SASZombieAssaultTD.Engine.UI.HUDPanels;

namespace SASZombieAssaultTD.Engine.UI
{
    public class HUDManager
    {
        private Resources.Texture2D _hudTexture;
        private Resources.Texture2D _supportHUDTexture;

        private HUDLayout _layout;
        private readonly Dictionary<string, HUDTextureElement> _textureDict = new();
        private readonly Dictionary<string, HUDTextElement> _textDict = new();
        private readonly List<IHUDElement> _renderOrder = new();

        private readonly TextureManager _textureManager;

        private Rectangle cashRect;
        private Rectangle livesRect;

```

```
62          private Rectangle cashPlusRect;
63          private Rectangle livesPlusRect;
64          private Rectangle livesMinusRect;
65          private Rectangle cashMinusRect;
66
67          public System.Drawing.Color ButtonFlashColor = System.Drawing.Color.Crimson;
68          private float _buttonFlashTimer = 0f;
69          protected const float BUTTON_FLASH_TIME = 0.10f;
70
71          private PlayerState _playerState;
72
73          public static HUDManager Instance { get; private set; }
74
75          // NEW FINALIZER PIPELINE
76          private HUDPanelFinalizer_Manager _finalizer;
77          private HUDPanel_CashUpdate _cashPanel;
78          //private HUDPanel_LivesUpdate _livesPanel;
79
80          public HUDPanel_CashUpdate CashPanel => _cashPanel;
81
82          public HUDManager(TextureManager textureManager)
83          {
84              _textureManager = textureManager ?? throw new ArgumentNullException(nameof(
                textureManager));
85              Instance = this;
86          }
87
88          //
            ============================================================================
            ====================
89          // LOAD HUD
90          //
            ============================================================================
            ====================
91          public void Load(string jsonPath)
92          {
93              // Load hitboxes from HUDConfigManager
94              var cashPanelConfig = HUDConfigManager.GetPanel("cash_panel");
95              if (cashPanelConfig != null)
96              {
97                  var cx = cashPanelConfig.crosshair?.centerX ?? 122;
98                  var cy = cashPanelConfig.crosshair?.centerY ?? 553;
99                  cashPlusRect = new Rectangle(
100                     (int)(float)(cx - 10),
101                     (int)(float)(cy - 10),
102                     (int)(float)20,
103                     (int)(float)20);
104             }
105             else
106             {
107                 cashPlusRect = new Rectangle(
108                     (int)(float)112,
109                     (int)(float)543,
110                     (int)(float)20,
111                     (int)(float)20);
112             }
113
114             var livesPanelConfig = HUDConfigManager.GetPanel("lives_panel");
115             if (livesPanelConfig != null)
116             {
117                 livesPlusRect = new Rectangle(
118                     (int)(float)(livesPanelConfig.x + livesPanelConfig.width - 10),
119                     (int)(float)(livesPanelConfig.y + 5),
120                     (int)(float)10,
121                     (int)(float)10
122                 );
123             }
124             else
125             {
```

```
126                 livesPlusRect = new Rectangle(
127                     (int)(float)230,
128                     (int)(float)550,
129                     (int)(float)10,
130                     (int)(float)10);
131             }
132 
133             var options = new JsonSerializerOptions { PropertyNameCaseInsensitive = true
                };
134 
135             if (File.Exists(jsonPath))
136             {
137                 var json = File.ReadAllText(jsonPath);
138                 _layout = JsonSerializer.Deserialize<HUDLayout>(json, options) ?? new
                    HUDLayout();
139             }
140             else
141             {
142                 _layout = new HUDLayout();
143             }
144 
145             _textureDict.Clear();
146             _textDict.Clear();
147             _renderOrder.Clear();
148 
149             // Load textures
150             foreach (var tex in _layout.Images)
151             {
152                 _textureDict[tex.Id] = tex;
153 
154                 string path = tex.Path;
155                 if (tex.Id == "hud_surface")
156                     path = "Assets/UI/HUD.png";
157                 else if (tex.Id == "support_hud")
158                     path = "Assets/UI/SupportHUD.png";
159 
160                 tex.Texture = _textureManager.Get(path);
161 
162                 if (tex.Id == "hud_surface")
163                     _hudTexture = (Resources.Texture2D)tex.Texture;
164                 else if (tex.Id == "support_hud")
165                     _supportHUDTexture = (Resources.Texture2D)tex.Texture;
166             }
167 
168             // Load text elements
169             foreach (var txt in _layout.Text)
170             {
171                 _textDict[txt.Id] = txt;
172 
173                 if (txt.Id == "cash_text")
174                     txt.Value = $"{((PlayerState)PlayerSystem.Instance?.State)?.Cash ??
                        0}";
175                 else if (txt.Id == "lives_text")
176                     txt.Value = $"{((PlayerState)PlayerSystem.Instance?.State)?.Lives ??
                        0}";
177                 else if (txt.Id == "waves_text")
178                     txt.Value = $"1/40";
179             }
180 
181             //
                ============================================================================
                ========================
182             // CREATE FINALIZER PIPELINE
183             //
                ============================================================================
                ========================
184             _cashPanel = new HUDPanel_CashUpdate();
185             var control = new HUDPanelFinalizer_Control();
186             var parser = new HUDPanelFinalizer_ColorParser();
```

```
            var resolver = new HUDPanelFinalizer_Resolver();
            var builder = new HUDPanelFinalizer_UIStateBuilder();

            _finalizer = new HUDPanelFinalizer_Manager(
                this,
                _cashPanel,
                control,
                parser,
                resolver,
                builder
            );

            _finalizer.Initialize();
            _renderOrder.Add(_cashPanel);

            // Build render order
            if (_layout.RenderOrder != null)
            {
                foreach (var id in _layout.RenderOrder)
                {
                    if (_textureDict.TryGetValue(id, out var tex))
                        _renderOrder.Add(tex);
                    else if (_textDict.TryGetValue(id, out var txt))
                        _renderOrder.Add(txt);
                }
            }

            foreach (var tex in _layout.Images.Where(i => !_renderOrder.Contains(i)).
            OrderBy(i => i.Layer))
                _renderOrder.Add(tex);

            foreach (var txt in _layout.Text.Where(t => !_renderOrder.Contains(t)).
            OrderBy(t => t.Layer))
                _renderOrder.Add(txt);

            if (!_renderOrder.Contains(_cashPanel))
            {
                _renderOrder.Insert(0, _cashPanel);
            }
            else
            {
                _renderOrder.Remove(_cashPanel);
                _renderOrder.Insert(0, _cashPanel);
            }

            DistributeFinalizerColors();
        }

        //
        ============================================================================
        ==================
        // UPDATE HUD
        //
        ============================================================================
        ==================
        public void Update(float deltaTime)
        {
            try
            {
                if (_buttonFlashTimer > 0)
                {
                    _buttonFlashTimer -= deltaTime;
                    if (_buttonFlashTimer <= 0)
                        ResetButtonFlash();
                }

                if (InputSystem.IsMouseButtonJustPressed(0))
                {
                    var mouse = InputSystem.GetMousePosition();
```

```
250
251                     // FIXED: Using explicit System.Math.Round to safely convert float
                        screen coordinates to a valid System.Drawing.Point
252                     int roundedX = (int)System.Math.Round((double)mouse.X);
253                     int roundedY = (int)System.Math.Round((double)mouse.Y);
254                     System.Drawing.Point mp = new System.Drawing.Point(roundedX, roundedY
                        );
255
256                     PlayerState state = (PlayerState)PlayerSystem.Instance.State;
257
258                     if (cashPlusRect.Contains(mp))
259                     {
260                         state.Cash += 1;
261                         TriggerButtonFlash();
262                         SnapshotIntegration.EnsureTodaySnapshot();
263                     }
264                     else if (livesPlusRect.Contains(mp))
265                     {
266                         state.Lives += 1;
267                         TriggerButtonFlash();
268                         SnapshotIntegration.EnsureTodaySnapshot();
269                     }
270                 }
271
272
273
274                 PlayerState ps = (PlayerState)PlayerSystem.Instance.State;
275
276                 UpdateText("cash_text", $"${ps.Cash}");
277                 UpdateText("lives_text", $"{ps.Lives}");
278                 UpdateText("waves_text", "1/40");
279             }
280             catch (Exception ex)
281             {
282                 DLogger.Log($"Error: HUDManager: Update failed: {ex.Message}");
283                 throw;
284             }
285         }
286
287         //
            ============================================================================
            ==================
288         // DRAW HUD
289         //
            ============================================================================
            ==================
290         public void Draw(IDrawingContext context, int crosshairCenterX = 0, int
            crosshairCenterY = 0)
291         {
292             try
293             {
294                 if (_layout == null)
295                     return;
296
297                 // Correct: GetResolved() returns a ResolvedState object
298                 var resolved = _finalizer.GetResolved();
299
300                 int cx = resolved.CrosshairCenterX;
301                 int cy = resolved.CrosshairCenterY;
302                 int armLength = resolved.CrosshairArmLength;
303                 int thickness = resolved.CrosshairThickness;
304
305                 var crosshairColor = (_buttonFlashTimer > 0)
306                     ? ButtonFlashColor
307                     : (System.Drawing.Color)resolved.CrosshairColor;
308
309                 foreach (var element in _renderOrder)
310                     element.Draw(context);
311
```

```
                var crossColor = new Color(
                    (byte)(crosshairColor.R / 255f * 255),
                    (byte)(crosshairColor.G / 255f * 255),
                    (byte)(crosshairColor.B / 255f * 255),
                    (byte)(crosshairColor.A / 255f * 255)
                );

                int horizX = cx - armLength;
                int horizY = cy - thickness / 2;
                int horizW = armLength * 2;
                int horizH = thickness;

                context.FillRectangle(new Rectangle((int)(float)horizX,
                    (int)(float)horizY, (int)(float)horizW, (int)(float)horizH),
                    crossColor);

                int vertX = cx - thickness / 2;
                int vertY = cy - armLength;
                int vertW = thickness;
                int vertH = armLength * 2;

                context.FillRectangle(new Rectangle((int)(float)vertX,
                    (int)(float)vertY, (int)(float)vertW, (int)(float)vertH), crossColor
                    );
            }
            catch (Exception ex)
            {
                DLogger.Log($"Error: HUDManager: Draw failed: {ex.Message}");
                throw;
            }
        }

        //
        ============================================================================
        ====================
        // TEXT UPDATE
        //
        ============================================================================
        ====================
        public void UpdateText(string id, string newValue)
        {
            if (_textDict.TryGetValue(id, out var txt))
            {
                if (txt.Value != newValue)
                    txt.Value = newValue;
            }
        }

        //
        ============================================================================
        ====================
        // BUTTON FLASH
        //
        ============================================================================
        ====================
        public virtual void TriggerButtonFlash()
        {
            ButtonFlashColor = System.Drawing.Color.LightGreen;
            _buttonFlashTimer = BUTTON_FLASH_TIME;
        }

        protected virtual void ResetButtonFlash()
        {
        }

        //
        ============================================================================
        ====================
        // FINALIZER COLOR DISTRIBUTION
```

```
        //
        ============================================================================
        ==================
        private void DistributeFinalizerColors()
        {
            if (_finalizer == null)
                return;

            var resolved = _finalizer.GetResolved();

            foreach (var element in _renderOrder)
            {
                if (element is HUDPanel_CashUpdate cashPanel)
                    _finalizer.ApplyToPanel(cashPanel, resolved);
            }
        }
    }

    // Simple config types to match how HUDManager uses HUDConfigManager
    public class HUDPanelCrosshairConfig
    {
        public int centerX { get; set; }
        public int centerY { get; set; }
    }

    public class HUDPanelConfig
    {
        public string id { get; set; }
        public int x { get; set; }
        public int y { get; set; }
        public int width { get; set; }
        public int height { get; set; }
        public HUDPanelCrosshairConfig crosshair { get; set; }
    }

    public static class HUDConfigManager
    {
        // In a full implementation, this would load from JSON or a config asset.
        // For now, we return hard-coded defaults that match HUDManager's expectations.
        public static HUDPanelConfig GetPanel(string id)
        {
            switch (id)
            {
                case "cash_panel":
                    return new HUDPanelConfig
                    {
                        id = "cash_panel",
                        x = 112,
                        y = 543,
                        width = 20,
                        height = 20,
                        crosshair = new HUDPanelCrosshairConfig
                        {
                            centerX = 122,
                            centerY = 553
                        }
                    };

                case "lives_panel":
                    return new HUDPanelConfig
                    {
                        id = "lives_panel",
                        x = 220,
                        y = 545,
                        width = 20,
                        height = 10
                    };

                default:
```

```
436                     return null;
437                 }
438             }
439
440             internal static void LoadPanel(string v, HUDPanelFinalizer_Control control)
441             {
442                 throw new NotImplementedException();
443             }
444
445             internal static void SetCrosshair(string v, int crosshairCenterX, int
                crosshairCenterY, int crosshairArmLength, int crosshairThickness, System.Drawing.
                Color crosshairColor, bool useFlashColor, System.Drawing.Color flashColor)
446             {
447                 throw new NotImplementedException();
448             }
449
450             internal static void SetPanel(string v, int x, int y, int width, int height, int
                layer)
451             {
452                 throw new NotImplementedException();
453             }
454         }
455     }
456
457
```